PYTHON DEVELOPER CHEATSHEET

A QUICK REFERENCE GUIDE

AZEEM TELI

AZEEM TELI

Python Developer Cheatsheet

First edition

This book was professionally typeset on Reedsy.
Find out more at reedsy.com

Contents

I

Basics of Python

1

Introduction to Python

What is Python?

- A high-level, interpreted programming language.
- Created by **Guido van Rossum** in the late 1980s (yes, it's older than most memes).
- Famous for being easy to read, like English, but stricter—Python doesn't tolerate sloppy indentation.

Why Python?

- **Readable**: Code looks clean and beginner-friendly.
- **Flexible**: Used in web, data science, AI, automation, and even making games.
- **Huge Community**: If you get stuck, chances are someone on the internet cried about it before you.

How Python Works

- **Interpreted**: Runs line by line (no need for a separate compile step).
- **Portable**: Runs on Windows, macOS, Linux, Raspberry Pi, and probably your fridge if it runs Linux.

Running Python

1. Install Python from python.org.
2. Open terminal/command prompt and type:

```
python --version
```

1. If you see something like Python 3.12.2, you're ready.
2. Run Python interactively:

```
python
```

1. You'll see the **>>>** prompt.

First Python Program

```
print("Hello, World!")
```

Output:

```
Hello, World!
```

This is tradition—you haven't *really* learned Python until you've made your computer greet the world.

2

Python Syntax & Comments

Python Syntax Basics

- Python uses **indentation** (spaces) to define code blocks. No curly braces {} or semicolons ;.
- Indentation is not optional—mess it up, and Python will shout at you.

Example:

```
if True:
    print("Indented correctly")
```

Wrong indentation:

```
if True:
print("This will throw an error")  # ✕
```

Case Sensitivity

- print, Print, and PRINT are not the same. Stick to lowercase for keywords and functions.

Statements

- One statement per line.
- You *can* put multiple statements on one line with ;, but don't—it looks messy.

```
x = 5; y = 10; print(x + y)  # Works, but ugly
```

Comments in Python

- Comments are ignored by Python, but loved by humans.
- Start with # for single-line comments.

```
# This is a comment
print("Hello")  # Inline comment
```

- Multi-line comments are usually just multiple #, but you can also use triple quotes (”””” or ‘‘‘) as a workaround.

```
"""
This is not officially a multi-line comment,
But Python ignores it anyway.
"""
```

Quick Note

- Write comments for the **future you** (because the future you won't remember what the past you was thinking).

3

Variables & Data Types

What is a Variable?

- A variable is just a **name that points to a value**.
- You don't need to declare types—Python figures it out (lazy but smart).

```
x = 10        # integer
name = "Ali"  # string
price = 99.9  # float
```

Naming Rules

- Must start with a letter or underscore (_), not a number.
- Can't use keywords (if, for, class, etc.).
- Be descriptive, but not like writing an essay:

```
student_name = "Azeem"   # good
sn = "Azeem"             # okay-ish
this_is_the_name_of_a_student_variable = "Azeem"  # ✕
```

Data Types in Python

Numbers

- Integers (int): 5, -12
- Floats (float): 3.14, -0.5
- Complex (complex): 2 + 3j (used rarely unless you're an engineer with too much math).
- **Strings (str)**
- Text inside quotes.

```
msg = "Hello"
```

Boolean (bool)

True or False (note the capitals).

```
is_happy = True
```

None Type

Represents "nothing" (Python's version of a shrug).

```
data = None
```

Checking Data Type

```
x = 42
print(type(x))   # <class 'int'>
```

Type Conversion (Casting)

```
x = "10"
print(int(x))    # 10
print(float(x))  # 10.0
print(str(99))   # "99"
```

Fun Fact

> *Python variables are like sticky notes—you can peel off the old value and stick a new one anytime.*

```
x = 100
x = "Now I'm a string"
```

4

Type Conversion & Casting

Two Ways to Convert Types

- **Implicit Conversion (Type Casting by Python)**

Python automatically converts smaller types to bigger ones when needed.

```
num_int = 5        # int
num_float = 2.5    # float
result = num_int + num_float
print(result)       # 7.5 (float)
print(type(result)) # <class 'float'>
```

Python: *"Don't worry, I got this."*

- **Explicit Conversion (Manual Casting)**

You tell Python what type to convert to.

```
x = "42"
print(int(x))     # 42
```

```
print(float(x))   # 42.0
print(str(99))    # "99"
print(bool(1))    # True
print(bool(0))    # False
```

Common Casting Functions

- int() → to integer
- float() → to float
- str() → to string
- bool() → to boolean

A Tricky Note

Not all conversions work smoothly:

```
print(int("Hello"))   # ✕ ValueError
```

Python: *"Sorry, I can't turn words into numbers."*

5

Input and Output

Output with print()

The simplest way to show stuff on the screen.

```
print("Hello, Python!")
print(2 + 3)        # 5
```

Multiple values:

```
name = "Ali"
age = 20
print("Name:", name, "Age:", age)
```

String Formatting (Cleaner Output)

- **f-strings (Python 3.6+)**

```
name = "Ali"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

- **format() method**

```
print("My name is {} and I am {} years old.".format(name, age))
```

Input with input()

- Always returns a **string**, no matter what you type.

```
name = input("Enter your name: ")
print("Hello,", name)
```

Type Casting User Input

Since input is always a string, cast it when needed:

```
age = int(input("Enter your age: "))
print(f"In 5 years, you'll be {age + 5}")
```

Quick Tip

If your program breaks after typing a number, you probably forgot to convert input() to int or float.

6

Operators

Operators are special symbols that let you do math, compare values, and make decisions.

1. Arithmetic Operators

Used for basic math.

```
x, y = 10, 3

print(x + y)   # 13 (addition)
print(x - y)   # 7  (subtraction)
print(x * y)   # 30 (multiplication)
print(x / y)   # 3.333... (division, float)
print(x // y)  # 3 (floor division, chops decimals)
print(x % y)   # 1 (modulus, remainder)
print(x ** y)  # 1000 (exponent, 10³)
```

2. Comparison Operators

Used to compare values. Always return True or False.

```
x, y = 5, 8

print(x == y)   # False
print(x != y)   # True
print(x > y)    # False
print(x < y)    # True
print(x >= 5)   # True
print(y <= 8)   # True
```

3. Logical Operators

Used to combine conditions.

```
a, b = True, False

print(a and b)  # False (both must be True)
print(a or b)   # True  (at least one True)
print(not a)    # False (reverses value)
```

4. Assignment Operators

Used to assign values.

```
x = 10     # basic assignment
x += 5     # x = x + 5 → 15
x -= 3     # x = x - 3 → 12
x *= 2     # x = x * 2 → 24
x /= 4     # x = x / 4 → 6.0
x %= 5     # x = x % 5 → 1
x **= 3    # x = x ** 3 → 1
```

Quick Note

Python follows **operator precedence** (order of operations). Parentheses () always win if you're confused.

II

Data Structures

7

Strings (Basics + Common Methods)

What is a String?

- A sequence of characters inside quotes (' ' or " ").

```
text1 = 'Hello'
text2 = "World"
```

- Multi-line strings use triple quotes:

```
msg = """This
is a
multi-line string."""
```

String Basics

```
word = "Python"

print(word[0])     # P (indexing starts at 0)
print(word[-1])    # n (negative index → from end)
print(word[0:3])   # Pyt (slicing, 0 to 2)
print(len(word))   # 6 (length of string)
```

Strings are **immutable** → you can't change them directly.

```
word[0] = "J"   # ✕ Error
```

String Operations

```
print("Hello " + "World")   # Concatenation
print("Hi! " * 3)           # Repeat → Hi! Hi! Hi!
```

Check membership:

```
print("Py" in word)   # True
print("Java" not in word)  # True
```

Common String Methods

```
txt = "  python is fun  "

print(txt.upper())      # "  PYTHON IS FUN  "
print(txt.lower())      # "  python is fun  "
print(txt.title())      # "  Python Is Fun  "
print(txt.strip())      # "python is fun" (removes spaces)
print(txt.replace("fun", "awesome"))  # python is awesome
```

```
print(txt.split())      # ['python', 'is', 'fun']
```

Searching:

```
print(txt.find("is"))   # 9 (index of first match)
print(txt.count("n"))   # 2 (number of 'n')
```

f-Strings (Quick Formatting)

```
name = "Ali"
age = 22
print(f"My name is {name}, I am {age} years old.")
```

Quick Note

> *Strings behave like lists of characters—but unlike lists, they can't be modified directly.*

8

Lists (Basics + Common Methods)

What is a List?

- A **collection of items**, ordered and changeable (mutable).
- Defined with square brackets [].

```
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.5, True]
```

Accessing Items

```
print(fruits[0])     # apple
print(fruits[-1])    # cherry (last item)
print(fruits[0:2])   # ['apple', 'banana'] (slicing)
```

Changing Items

```
fruits[1] = "mango"   # replace banana
print(fruits)  # ['apple', 'mango', 'cherry']
```

Adding Items

```
fruits.append("orange")     # add to end
fruits.insert(1, "grape")   # insert at index 1
print(fruits)
```

Removing Items

```
fruits.remove("apple")   # remove by value
fruits.pop()             # remove last item
fruits.pop(0)            # remove by index
del fruits[1]            # delete by index
fruits.clear()           # remove all items
```

Common List Methods

```
nums = [4, 2, 9, 1]

nums.sort()       # [1, 2, 4, 9] (ascending)
nums.reverse()    # [9, 4, 2, 1] (reverses order)
print(len(nums))  # 4
print(max(nums))  # 9
```

```
print(min(nums))  # 1
print(sum(nums))  # 16
```

List Operations

```
a = [1, 2, 3]
b = [4, 5]
print(a + b)     # [1, 2, 3, 4, 5] (concatenation)
print(a * 2)     # [1, 2, 3, 1, 2, 3] (repetition)
```

Check membership:

```
print(2 in a)    # True
print(10 not in a)  # True
```

Nested Lists

```
matrix = [[1, 2], [3, 4], [5, 6]]
print(matrix[1][0])   # 3
```

Quick Note

> *Lists are like Python's shopping bags—you can throw in anything, mix numbers with strings, and even other lists.*

9

Tuples

What is a Tuple?

- A **collection of items** just like a list, but **immutable** (cannot be changed).
- Defined with parentheses ().

```
t = (1, 2, 3, "hello")
```

Tuple Basics

```
print(t[0])     # 1
print(t[-1])    # hello
print(len(t))   # 4
```

- Tuples can hold mixed data types.
- Tuples are **faster** than lists (since they can't change).

Single Item Tuple

```
t1 = (5,)    # note the comma → tuple
t2 = (5)     # just an int
```

Tuple Operations

```
t = (1, 2, 3, 4, 2)

print(t.count(2))    # 2 (how many times 2 appears)
print(t.index(3))    # 2 (first index of 3)
```

Concatenation and repetition:

```
a = (1, 2)
b = (3, 4)
print(a + b)    # (1, 2, 3, 4)
print(a * 2)    # (1, 2, 1, 2)
```

Tuple Packing & Unpacking

```
person = ("Ali", 22, "Student")

name, age, role = person
print(name)  # Ali
```

When to Use Tuples?

- When you want data to stay constant (e.g., coordinates, settings).
- Think of them as "read-only lists."

10

Sets

What is a Set?

- A collection of **unique, unordered items**.
- Defined with curly braces {} or set().

```
s = {1, 2, 3, 3, 2}
print(s)   # {1, 2, 3} (duplicates removed)
```

Creating Sets

```
empty = set()   # correct way to create empty set
not_set = {}    # this is a dictionary, not a set
```

Basic Operations

```
fruits = {"apple", "banana", "cherry"}

fruits.add("orange")    # add item
fruits.remove("banana") # remove item (error if not found)
fruits.discard("pear")  # remove safely (no error)
fruits.pop()            # removes random item
fruits.clear()          # empty set
```

Set Operations (Math-like)

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)   # union → {1, 2, 3, 4, 5}
print(a & b)   # intersection → {3}
print(a - b)   # difference → {1, 2}
print(a ^ b)   # symmetric difference → {1, 2, 4, 5}
```

Membership Test

```
print(2 in a)     # True
print(10 not in a) # True
```

Quick Note

- Sets are **unordered** → no indexing (`s[0]` ✕).
- Use sets when you need **unique items** and **fast lookups**.

11

Dictionaries

What is a Dictionary?

- A **collection of key–value pairs**.
- Keys must be unique and immutable (strings, numbers, tuples).
- Values can be anything.

```
student = {
    "name": "Ali",
    "age": 21,
    "grade": "A"
}
```

Accessing Values

```
print(student["name"])        # Ali
print(student.get("age"))     # 21
print(student.get("roll", "N/A"))  # safer (returns "N/A" if key
not found)
```

Adding / Updating Items

```
student["age"] = 22          # update
student["city"] = "Lahore"   # add new key-value pair
```

Removing Items

```
student.pop("grade")     # remove by key
student.popitem()        # removes last inserted pair
del student["city"]      # delete by key
student.clear()          # empty dictionary
```

Dictionary Methods

```
info = {"a": 1, "b": 2, "c": 3}

print(info.keys())     # dict_keys(['a', 'b', 'c'])
print(info.values())   # dict_values([1, 2, 3])
print(info.items())    # dict_items([('a', 1), ('b', 2), ('c', 3)])
```

Looping through the Dictionary

```
for key, value in info.items():
    print(key, ":", value)
```

Nesting Dictionaries

```
students = {
    "101": {"name": "Ali", "age": 21},
    "102": {"name": "Sara", "age": 22}
}
print(students["101"]["name"])  # Ali
```

Quick Note

> *Dictionaries are like Python's version of a* ***real-world dictionary*** *→ you look up a* word *(key) and get its* meaning *(value).*

III

Flow Control

12

Conditional Statements

Why Conditionals?

They let Python **make decisions**. Without them, programs would just run in a straight line like a robot with no brain.

Basic if Statement

```
x = 10

if x > 5:
    print("x is greater than 5")
```

if-else

```
age = 18

if age >= 18:
    print("You are an adult")
```

```
else:
    print("You are a minor")
```

if-elif-else

```
score = 75

if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
else:
    print("Grade D")
```

Nested if

```
num = 10

if num > 0:
    if num % 2 == 0:
        print("Positive Even")
```

Short Hand if (One-Liner)

```
x = 5
print("Yes") if x > 0 else print("No")
```

Quick Note

- Indentation is critical → Python decides blocks by spaces, not braces.
- Use elif instead of chaining multiple ifs for cleaner logic.

13

Loops

Why Loops?

Loops let Python **repeat tasks** without you copy–pasting code like a rookie.

for Loop

- Used to iterate over a sequence (string, list, tuple, dictionary, range, etc.).

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Using range():

```
for i in range(5):
    print(i)   # 0 to 4
```

With start and step:

```
for i in range(2, 10, 2):
    print(i)   # 2, 4, 6, 8
```

while Loop

- Runs **as long as** a condition is True.

```
count = 0
while count < 5:
    print(count)
    count += 1
```

⚠ Be careful—forgetting to update the condition = infinite loop (Python will never stop until your computer cries).

Loop with else

Yes, Python lets you use else with loops. It runs if the loop wasn't broken by break.

```
for i in range(3):
    print(i)
else:
    print("Loop finished")
```

Loop Control Statements

```
for i in range(5):
    if i == 2:
        continue   # skip this iteration
    if i == 4:
```

```
        break      # exit loop completely
    print(i)
```

Quick Note

- Use for when you know how many times to repeat.
- Use while when you repeat until a condition changes.

14

Loop Control Statements

1. break

- Exits the loop immediately, even if the condition isn't finished.

```
for i in range(5):
    if i == 3:
        break
    print(i)
# Output: 0 1 2
```

2. continue

- Skips the current iteration and moves to the next one.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

```
# Output: 0 1 3 4
```

3. pass

- Does nothing. Just a placeholder to keep code valid.

```
for i in range(3):
    if i == 1:
        pass    # TODO: write code later
    print(i)
```

Quick Note

- **break** → stop the loop.
- **continue** → skip and move on.
- **pass** → a polite "do nothing" statement (useful for stubs).

IV

Functions & Modules

15

Functions

What is a Function?

- A reusable block of code that performs a task.
- Defined with the def keyword.
- Functions keep your code DRY (*Don't Repeat Yourself*).

Defining a Function

```
def greet():
    print("Hello from Python!")
```

Calling a Function

```
greet()   # Output: Hello from Python!
```

Functions with Parameters

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ali")   # Hello, Ali!
greet("Sara")  # Hello, Sara!
```

Functions with Return Values

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)   # 8
```

Default Parameters

```
def greet(name="Guest"):
    print(f"Welcome, {name}!")

greet()        # Welcome, Guest!
greet("Azeem") # Welcome, Azeem!
```

Quick Note

- def = define once, call many times.
- Functions help avoid copy–paste coding (a.k.a. programmer's nightmare).

16

Function Arguments

1. Positional Arguments

- Passed in the order they are defined.

```
def greet(name, age):
    print(f"{name} is {age} years old")

greet("Ali", 22)
```

2. Default Arguments

- If no value is provided, a default is used.

```
def greet(name="Guest"):
    print(f"Hello, {name}")

greet()         # Hello, Guest
greet("Sara")   # Hello, Sara
```

3. Keyword Arguments

- Specify arguments by name → order doesn't matter.

```
def intro(name, city):
    print(f"{name} lives in {city}")

intro(city="Lahore", name="Azeem")
```

*4. *args (Arbitrary Positional Arguments)*

- Collects extra positional arguments into a tuple.

```
def add(*numbers):
    print(sum(numbers))

add(1, 2, 3)        # 6
add(5, 10, 15, 20)  # 50
```

*5. **kwargs (Arbitrary Keyword Arguments)*

- Collects extra keyword arguments into a dictionary.

```
def profile(**info):
    print(info)

profile(name="Ali", age=22, city="Karachi")
# {'name': 'Ali', 'age': 22, 'city': 'Karachi'}
```

Mixing Them (Order Matters)

positional → default → *args → **kwargs

```
def demo(x, y=10, *args, **kwargs):
    print(x, y, args, kwargs)

demo(5, 20, 1, 2, 3, name="Ali", city="Lahore")
# 5 20 (1, 2, 3) {'name': 'Ali', 'city': 'Lahore'}
```

Quick Note

- *args = pack extra values into a tuple.
- **kwargs = pack extra named values into a dictionary.
- Order is strict, or Python throws a tantrum.

17

Return Values

What is a Return Value?

- Functions can **send data back** using return.
- Without return, functions give back None by default.

Basic Example

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)   # 8
```

Returning Multiple Values

- Python can return multiple values (as a tuple).

```
def get_person():
    return "Ali", 22, "Lahore"

name, age, city = get_person()
print(name, age, city)
```

Returning Early

- return also stops function execution immediately.

```
def check(num):
    if num < 0:
        return "Negative"
    return "Positive"

print(check(-5))  # Negative
```

No Return = None

```
def greet():
    print("Hello!")

result = greet()
print(result)   # None
```

Quick Note

- return makes functions reusable and flexible.
- Think of it like a **vending machine**: you put in arguments (coins), and return gives you the output (snack).

18

Lambda Functions

What is a Lambda Function?

- A **small anonymous function** that can have any number of arguments, but only **one expression**.
- Also called **"throwaway" functions** because you don't need to name them.

```
square = lambda x: x ** 2
print(square(5))   # 25
```

Syntax

```
lambda arguments: expression
```

- Example:

```
add = lambda a, b: a + b
print(add(3, 7))   # 10
```

Using Lambda Without Assignment

```
print((lambda x, y: x * y)(4, 5))  # 20
```

Common Use Case: map(), filter(), reduce()

```
nums = [1, 2, 3, 4, 5]

# map → apply function to each item
squared = list(map(lambda x: x**2, nums))
print(squared)   # [1, 4, 9, 16, 25]

# filter → pick items that satisfy condition
even = list(filter(lambda x: x % 2 == 0, nums))
print(even)      # [2, 4]
```

Quick Note

- Lambda functions are **one-liners**.
- Great for short, temporary functions, but **don't overuse** — named functions are more readable.

19

Importing & Using Modules

What is a Module?

- A **module** is a Python file (.py) containing functions, classes, or variables.
- Helps **organize code** and **reuse** functionality.

Import a Module

```
import math

print(math.sqrt(16))   # 4.0
```

- Python searches in **standard library**, then installed packages, then current directory.

Import Specific Functions/Variables

```
from math import pi, ceil

print(pi)     # 3.141592653589793
print(ceil(4.2))  # 5
```

Import with Alias

```
import numpy as np

arr = np.array([1, 2, 3])
print(arr)
```

- Aliases make long module names shorter and more readable.

Your Own Module

- Create a file my_module.py:

```
def greet(name):
    return f"Hello, {name}"
```

- Import in another file:

```
import my_module

print(my_module.greet("Ali"))
```

Quick Note

- Modules = reusable code blocks.
- Python comes with **hundreds of built-in modules** (math, random, datetime, os, sys...).

20

Built-in Modules

1. math Module

- Provides mathematical functions.

```
import math

print(math.sqrt(16))    # 4.0
print(math.ceil(4.2))   # 5
print(math.floor(4.7))  # 4
print(math.pi)          # 3.141592653589793
print(math.factorial(5)) # 120
```

2. random Module

- Generate random numbers or choose random items.

```
import random

print(random.randint(1, 10))   # random integer between 1 and 10
print(random.choice(["apple", "banana", "cherry"]))  # random
element
print(random.shuffle([1,2,3,4]))  # shuffle list in-place
```

3. datetime Module

- Work with dates and times.

```
import datetime

now = datetime.datetime.now()
print(now)                  # current date & time
print(now.year, now.month)  # 2025 8
print(now.strftime("%A"))   # weekday name
```

4. os Module

- Interact with the operating system.

```
import os

print(os.getcwd())       # current working directory
os.mkdir("test_folder")  # create a folder
os.listdir()             # list files/folders
```

5. sys Module

- Access system-specific parameters and functions.

```
import sys

print(sys.version)   # Python version
print(sys.platform)  # OS platform
```

Quick Note

- Python comes with **hundreds of built-in modules** — check Python docs for more.
- Using modules = **less reinventing the wheel**.

V

Advanced Basics

21

File Handling

Opening a File

```
file = open("example.txt", "mode")
```

- Modes:
- “r” → read (default)
- “w” → write (creates new or overwrites)
- “a” → append (add at the end)
- “rb” / “wb” → read/write in binary

Writing to a File

```
with open("example.txt", "w") as f:
    f.write("Hello, Python!\n")
    f.write("This is a second line.")
```

- with automatically **closes the file**.

Appending to a File

```
with open("example.txt", "a") as f:
    f.write("\nAppending a new line.")
```

Reading a File

```
# Read entire content
with open("example.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line
with open("example.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Common File Methods

```
f = open("example.txt", "r")
print(f.readline())     # read first line
print(f.readlines())    # read all lines as a list
f.close()               # manually close file
```

Quick Note

- Always **close files** or use with to avoid resource leaks.
- "w" overwrites → be careful, don't lose important data.

22

Error & Exception Handling

Why Exception Handling?

- Prevents your program from crashing when something goes wrong.
- Lets you **handle errors gracefully**.

Basic try / except

```
try:
    x = int(input("Enter a number: "))
    print(10 / x)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input, enter a number!")
```

- Multiple except blocks can catch different types of errors.

Catch All Exceptions

```
try:
    risky_code()
except Exception as e:
    print("Something went wrong:", e)
```

else Block

- Runs if **no exception** occurs.

```
try:
    x = int(input("Enter a number: "))
except:
    print("Error!")
else:
    print("Success! Number is", x)
```

finally Block

- Always runs, no matter what.
- Good for **clean-up tasks**.

```
try:
    f = open("file.txt")
    data = f.read()
except FileNotFoundError:
    print("File not found!")
finally:
    f.close()
```

```
    print("File closed!")
```

Quick Note

- try = risky code
- except = handle the mistake
- else = happy path
- finally = cleanup crew

23

Classes & Objects

What is OOP?

- **Object-Oriented Programming** = organizing code using **objects** that represent real-world things.
- Main concepts: **Class, Object, Attributes, Methods**.

Defining a Class

```
class Person:
    def __init__(self, name, age):
        self.name = name   # attribute
        self.age = age

    def greet(self):       # method
        print(f"Hello, my name is {self.name} and I am {self.age}
        years old.")
```

- ___init___ → constructor, runs when an object is created.
- self → refers to the instance itself.

Creating Objects

```
p1 = Person("Ali", 22)
p2 = Person("Sara", 20)
```

Accessing Attributes & Methods

```
print(p1.name)   # Ali
print(p2.age)    # 20

p1.greet()       # Hello, my name is Ali and I am 22 years old.
```

Modifying Attributes

```
p1.age = 23
print(p1.age)    # 23
```

Quick Note

- Class = blueprint
- Object = house built from a blueprint
- Attributes = properties
- Methods = actions

24

Inheritance & Polymorphism

1. Inheritance

- Allows a class (**child**) to **reuse attributes and methods** of another class (**parent**).
- Avoids repeating code.

```
class Animal:
    def speak(self):
        print("Some generic sound")

class Dog(Animal):   # Dog inherits from Animal
    def speak(self):
        print("Woof woof!")

a = Animal()
a.speak()    # Some generic sound

d = Dog()
d.speak()    # Woof woof!
```

- Child class can **override parent methods** (as seen with speak).

2. Polymorphism

- Same **method name** behaves differently depending on the object.
- Makes code flexible and readable.

```
class Cat(Animal):
    def speak(self):
        print("Meow!")

animals = [Dog(), Cat(), Animal()]

for pet in animals:
    pet.speak()
# Output:
# Woof woof!
# Meow!
# Some generic sound
```

Quick Notes

- **Inheritance** = code reuse
- **Polymorphism** = same action, different behavior
- Python supports **multiple inheritance** too, but keep it simple as a beginner.

25

Python Packages & pip

What is a Package?

- A **package** is a collection of Python modules organized in folders.
- Helps **organize and share code**.

Example folder structure:

```
mypackage/
    __init__.py
    module1.py
    module2.py
```

Installing Packages with pip

- pip = Python's package manager.

```
pip install package_name      # install a package
pip install --upgrade package_name  # upgrade a package
```

```
pip uninstall package_name    # remove a package
pip list                      # list installed packages
```

Importing Installed Packages

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
```

- You can also import specific functions:

```
from math import sqrt
print(sqrt(16))  # 4.0
```

Creating Your Own Package

1. Create a folder with an ___init___.py file.
2. Add modules (.py files) inside.
3. Import in other projects:

```
from mypackage import module1
module1.my_function()
```

Quick Notes

- Packages = organized modules
- pip = install, upgrade, remove external libraries
- Python has **PyPI (Python Package Index)** with thousands of packages.

VI

Useful Tools & Extras

26

List Comprehensions

What is a List Comprehension?

- A **compact way to create lists** using a single line of code.
- Often replaces loops for simple tasks.

Basic Syntax

```
[expression for item in iterable]
```

Example:

```
squares = [x**2 for x in range(5)]
print(squares)   # [0, 1, 4, 9, 16]
```

With Condition (Filter)

```
evens = [x for x in range(10) if x % 2 == 0]
print(evens)    # [0, 2, 4, 6, 8]
```

Nested Loops

```
matrix = [[1,2,3], [4,5,6]]
flatten = [num for row in matrix for num in row]
print(flatten)  # [1, 2, 3, 4, 5, 6]
```

Quick Notes

- Faster and cleaner than normal loops for simple list operations.
- Don't overuse for **complex logic** — regular loops are more readable.

27

Dictionary & Set Comprehensions

1. Set Comprehensions

- Similar to lists, but results are **unique and unordered**.

```
nums = [1, 2, 2, 3, 4]
squared_set = {x**2 for x in nums}
print(squared_set)   # {16, 1, 4, 9} (order may vary)
```

- With a condition:

```
evens = {x for x in range(10) if x % 2 == 0}
print(evens)    # {0, 2, 4, 6, 8}
```

2. Dictionary Comprehensions

- Create dictionaries in a **compact way**.

```
squares = {x: x**2 for x in range(5)}
print(squares)  # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

- With condition:

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares)  # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Quick Notes

- Set comprehension = {expression for item in iterable if condition}
- Dict comprehension = {key_expression: value_expression for item in iterable if condition}
- Use comprehensions for **compact, readable code**, but avoid overly complex expressions.

28

Iterators & Generators (Basic Intro)

1. Iterators

- An **iterator** is an object that can be **looped through** using next().
- Any object that implements ___iter___() and ___next___() is an iterator.

```
nums = [1, 2, 3]
it = iter(nums)

print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
# next(it) again → StopIteration
```

- **for loops** automatically use iterators.

2. Generators

- Generators are **lazy iterators** → they produce items **on the fly**, saving memory.
- Created with **functions + yield**.

```
def my_gen():
    for i in range(5):
        yield i

gen = my_gen()
print(next(gen))  # 0
print(next(gen))  # 1
```

- Use in loops:

```
for val in my_gen():
    print(val)
```

Generator Expressions

- Like list comprehensions, but **lazy** (don't store all items in memory).

```
squares = (x**2 for x in range(5))
for sq in squares:
    print(sq)
```

Quick Notes

- Iterators → general way to loop through objects.
- Generators → produce items one by one, **efficient for large datasets**.
- Use yield instead of return inside generators.

29

Working with Virtual Environments

What is a Virtual Environment?

- An isolated Python environment where you can **install packages without affecting the global Python installation**.
- Helps avoid version conflicts between projects.

Create a Virtual Environment

```
# Python 3
python -m venv myenv
```

- myenv = name of your virtual environment.

Activate Virtual Environment

- **Windows**

```
myenv\Scripts\activate
```

- **macOS / Linux**

```
source myenv/bin/activate
```

- After activation, you'll see (myenv) in the terminal.

Deactivate Virtual Environment

```
deactivate
```

Installing Packages Inside Virtual Environment

```
pip install package_name
pip list          # list installed packages
pip freeze        # output packages for requirements.txt
```

Quick Notes

- Each project can have its **own dependencies** in separate virtual environments.
- Think of it as a **sandbox** for your Python projects.

30

Best Practices & Coding Style (PEP 8 Basics)

1. Code Layout & Indentation

- Use **4 spaces** per indentation level.

```
def greet(name):
    print(f"Hello, {name}")
```

- Avoid tabs → stick to spaces.

2. Naming Conventions

- **Variables / functions** → snake_case
- **Classes** → CamelCase
- **Constants** → UPPER_CASE

```
user_name = "Ali"
MAX_LIMIT = 100

class Person:
    pass
```

3. Line Length & Spacing

- Max **79 characters per line**.
- Add **blank lines** between functions/classes for readability.

```
def func1():
    pass


def func2():
    pass
```

4. Comments & Docstrings

- Use comments to explain **why**, not **what**.
- Docstrings for functions/classes:

```
def add(a, b):
    """Return the sum of a and b"""
    return a + b
```

5. Imports

- One per line, at the **top of the file**.
- Standard → Third-party → Local

```
import os
import requests
from mymodule import func
```

6. Avoid Bad Practices

- Don't use * imports → messy namespace
- Don't shadow built-in names (list, str, etc.)
- Keep code **readable and maintainable**

Quick Notes

- PEP 8 = Python's **official style guide**
- Readable code = easier debugging + collaboration

31

Quick Reference Section

Strings

- Python String Methods: https://docs.python.org/3/library/stdtypes.html#string-methods

Lists

- Python List Methods: https://docs.python.org/3/tutorial/datastructures.html#more-on-lists

Dictionaries

- Python Dictionary Methods: https://docs.python.org/3/tutorial/datastructures.html#dictionaries

Sets

- Python Set Methods: https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset

File Handling

- Reading & Writing Files: https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

Built-in Functions

- Full Built-ins Reference: https://docs.python.org/3/library/functions.html

Modules & Packages

- Python Modules: https://docs.python.org/3/tutorial/modules.html
- Python Packages: https://packaging.python.org/en/latest/

Error & Exception Handling

- Exceptions: https://docs.python.org/3/tutorial/errors.html

Object-Oriented Programming (OOP)

- Classes & Objects: https://docs.python.org/3/tutorial/classes.html

Iterators & Generators

- Iterators: https://docs.python.org/3/tutorial/classes.html#iterators
- Generators: https://docs.python.org/3/tutorial/classes.html#generators

Virtual Environments

- venv Module: https://docs.python.org/3/library/venv.html

PEP 8 & Best Practices

- PEP 8 Style Guide: https://pep8.org/

About the Author

Azeem Teli is a Python developer and writer known for making complex programming topics approachable and engaging. He is the creator behind the blog "PyZilla" and contributes to several technical publications, including "**Data Engineer Things**" and "Better Programming." His writing often adopts a conversational tone, likening coding to casual "chai-time gossip," with the aim of demystifying concepts in Python, Django, and web development for his readers.

Azeem's content is particularly popular among developers seeking to enhance their skills without feeling overwhelmed. He focuses on practical tutorials, clean code practices, and insights into tools like SQLAlchemy and Google's Project IDX. His work has been featured in major publications, including "Python in Plain English" and "JavaScript in Plain English."

In addition to his technical writing, Azeem is active on Medium, where he shares personal reflections on his journey as a developer. His articles often blend humor and storytelling, reflecting his belief that coding doesn't have to be boring.

For those interested in learning Python or exploring web development, Azeem Teli's work offers a refreshing and accessible perspective.

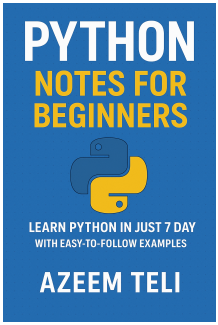
You can connect with me on:

https://azeemteli.com

Subscribe to my newsletter:

https://bit.ly/azeemteli

Also by Azeem Teli



Python Notes For Beginners

https://bit.ly/azeemteli

Learn Python in Just 7 Days with Easy-to-Follow Examples